

Homework 3

Meenu Ravi

March 6 2026

1 Option 1

Use the code you have chosen for your semester project to implement the Method of Manufactured Solutions. Choose the form of the manufactured solution according to the guidelines given in the lecture notes. Use your manufactured solution to specify any needed boundary and/or initial conditions (i.e., you do not need to verify your actual boundary and initial conditions in your code).

Using the code from <https://github.com/dsanmartin/ngen-kutral/blob/master/wildfire/numerical/time/integration.py>, I have implemented the method of manufactured solutions.

- Step 1: Write PDEs in the form:

$$u_t = \kappa \nabla u - \mathbf{v} \cdot \nabla u + f(u, \beta) \quad (1)$$

- Step 2: Choose the analytic form of the manufactured solution:

$$u(x, y, t) = (1 + t) \sin(\pi x) \sin(\pi y) \quad (2)$$

- Step 3: Operate the mathematical model onto the manufactured solution to obtain the analytic source term

$$S_u = u_t - \kappa \nabla u \quad (3)$$

$$S_u = \sin(\pi x) \sin(\pi y) + 2\kappa\pi^2(1 + t) \sin(\pi x) \sin(\pi y) \quad (4)$$

- Step 4: Obtain the modified form of the mathematical model by including the analytic source term:

$$u_t = \kappa \nabla u - \mathbf{v} \cdot \nabla u + S_u \quad (5)$$

2 Numerical solutions on at least four systematically-refined meshes

The four meshes chosen were 16,32,64, and 128. In addition to the L2 and the L-inf norms, the system response quantity (SRQ) chosen for this study is the maximum temperature. The error for this was calculated as the difference between the numerical and exact values. Below is the plot of the order of accuracy of these discretization error norms and the SRQs as a function of the discretization parameter h.

Mesh Nodes	L_2 Error	L_∞ Error	SRQ Error
16	0.00260611	0.00515526	0.00515526
32	0.00060983	0.00121653	0.00121653
64	0.00014764	0.00029509	0.00029509
128	0.00003633	0.00007265	0.00007265

Table 1: Discretization errors for different mesh sizes

This shows that as h decreases, the error decreases. Errors decrease by a factor of 4 when h is half-ed. This passes the order of accuracy test for this case. From this plot, it can be observed that the convergence

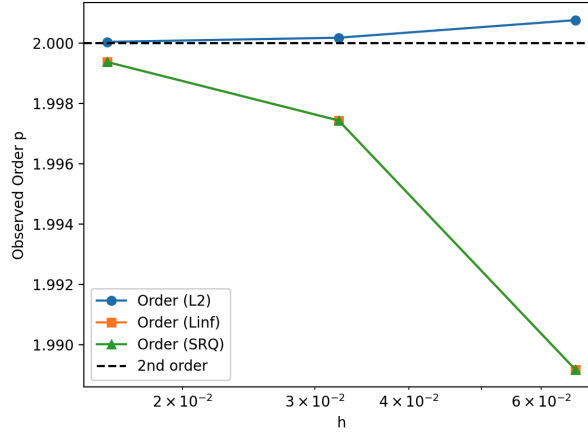


Figure 1: Order of accuracy of discretization error norms and the SRQ as a function of the discretization parameter h

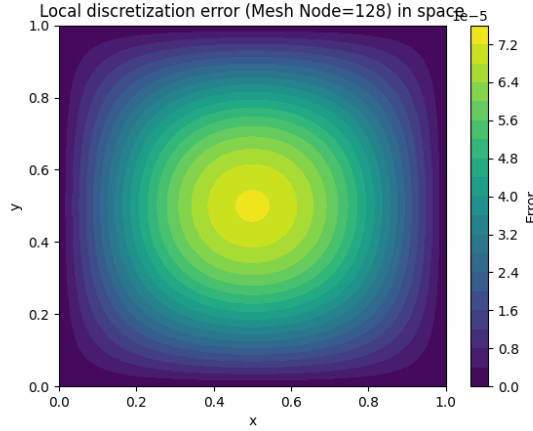


Figure 2: Plot of the local discretization error relative to exact solution

rates were approximately 2.0 for all grid sizes. These values align with the second -order accuracy of the finite difference discretization and therefore passes the order of accuracy test for the manufactured solution.

The local discretization error relative to the exact solution is largest at the center ($7.5e-5$) and decreases towards 0 at all 4 boundaries. This is consistent with the manufactured solution and satisfies the Dirichlet boundary conditions on all 4 sides. The contours are also smooth suggesting that the finite difference method behaviour is consistent throughout.

3 Code Coverage

The level of code coverage is low because there are several areas in which there is yet to have code for testing the capabilities. First, the solver can test multiple components such as diffusion, convection, and reaction. However, diffusion is the only one tested for the manufactured solution. Therefore, having other combinations (i.e., diffusion and convection or diffusion and reaction or all three, as well as the other permutations) can improve code coverage. Second, for space, finite difference method is tested, but the model accepts Fast Fourier Transform (FFT) method. Similarly, for time discretization, fourth order Runge-Kutta is tested but Euler method is not. Next, for the boundary conditions, additional boundary conditions such as periodic

boundaries still need to be tested. Lastly, the other dimensions (2-D is currently tested) can also be tested for dimensional coverage.

4 Round off Errors

For estimating the round-off error, I tried using float64 and increasing to np.longdouble, hoping for higher precision. However, on macOS, both defaulted to float64, resulting in a round-off estimate of 0. Therefore, I used float32 (single-precision) and float64 (double-precision) as the low and high precisions respectively. The simulation was run at both precisions with the same mesh (16) and time step, and the round-off error was estimated as the difference between the two solutions. The errors were:

- Round-off L_∞ : 2.62e-07
- Round-off L_2 : 1.15e-07

These values are consistent within machine precision, where sensitivity to round-off error is discernable.

5 Version Control Discussion

I used Github for my version control. The repository is found here: <https://github.com/meenuravi18/Verification-and-Validation-in-Scientific-Computing/tree/main/homework3>. I thought it was very easy to use and since I have more experience using Git before, I found it very convenient for me to use and save my work periodically. When I uploaded changes to the code, Github allowed me to see what was deleted in red and added

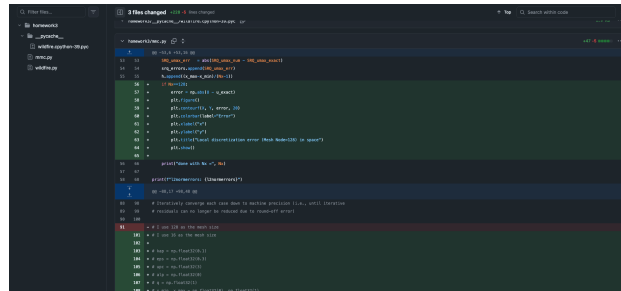


Figure 3: Example of Github view for examining different versions of the code

in green since the last commit. This allows changes to coding to be visually examined. It also highlights the number of files changes and the number of changes.